IDEA 3 — TestOrCode: Intent-Grounded Triage of CI Test Failures with Scoped Change Memory and Test-Integrity Checking

Final version. The core study uses free hosted APIs for all LLM, embedding and data access (Section 13). Fine-tuning is an optional extension described in Appendix A; nothing runs on a local machine.

1. One-line Concept

A multi-agent system for one narrow SE decision: when unit tests fail after a code change, is the **test outdated** (the behaviour was intentionally changed) or has the **code regressed**? The system grounds the decision in change-intent evidence, remembers approved behaviour changes with executable probes, and checks that its repair does not weaken the test suite.

2. Domain and Target Users

Domain

Software engineering (priority 3), specifically **CI test-failure triage and repair** in small Python libraries that use pytest. Flaky tests, build failures and non-Python projects are out of scope.

Target Users

- **Contributors and maintainers of small open-source Python libraries**, and student project teams in a software engineering course, whose CI turns red after a change.
- **The reviewing maintainer**, who receives either a patch with cited evidence or an escalation with a case summary.

Why This Domain Is Suitable for College Students

- We all use git and pytest, so no expert knowledge is needed to understand failures or verify labels.
- Ground truth comes from the code itself: real commit histories, real test diffs and executable tests.
- Everything runs in a Docker sandbox at no risk. No production systems are involved.

3. Concrete Problem

Problem Statement

1. When CI fails after a change, a developer has to decide which side is wrong: the test, because behaviour changed on purpose, or the code, because it regressed.
2. Coding agents are typically rewarded by "the tests pass", and there are public reports of agents editing, special-casing or deleting tests to get to green, though we have to verify the exact sources.

3. A wrong-side repair is costly in both directions. Editing the test when the code regressed **masks the bug** and permanently weakens the suite. Editing the code to satisfy an old test **silently reverts an intended change**.
4. The evidence needed to choose lives outside the failing output: the commit message, changelog, docs and earlier approved decisions. Docstrings and old tests often describe the *old* behaviour, so naive retrieval can support the wrong side.
5. Across a stream of commits, one deliberate change often breaks many tests over several commits. Handling each failure from scratch is inconsistent and expensive, but reusing memory too broadly makes a later real regression look like "the same intended change".
6. We therefore study a **decision-and-integrity problem**: can an agent choose the correct side per failing test, repair it without weakening the suite, and reuse approved changes only when the current behaviour actually matches them?
7. Scope: 3 small Python libraries, one failure type (assertion failures in unit tests), three outcomes (UPDATE_TEST, FIX_CODE, ESCALATE).
8. Success is measured against labels that are known by construction (see Section 4), not by "tests pass".

Domain Constraints

- **"Tests pass" is a gameable objective.** The evaluation has to measure test integrity, not just green builds.
- **Intent evidence is incomplete and sometimes stale**, so the system must be able to say "I can't tell" and escalate.
- **Mixed failures:** one change can produce both outdated tests and real regressions in the same CI run, so decisions are per failing test.
- **Cross-commit consistency:** the same approved change should be handled the same way later, at lower cost.
- **Safe editing:** only the correct side may be edited, and CI config, dependencies and `conftest.py` are off limits.
- **Reproducibility:** deterministic test runs and no leakage of future commits.
- **Budget:** a bounded number of tool calls and tokens per episode, within free API rate limits.

Why This Is a Real Problem

Test maintenance is a known cost in software projects, and test code has to be updated as the system under test evolves. The concern here is quieter. An assistant that makes CI green by the cheapest edit can degrade the suite without anyone noticing. We treat the size of this effect as an empirical question for our pilot.

Why a Normal LLM Is Insufficient

- It sees a traceback and a diff, but cannot run tests, inspect history or check its own patch.
- The path of least resistance is to change the assertion to match the new output. Nothing in the prompt forces it to look for evidence that the new output is *intended*.
- It has no memory of what maintainers approved in earlier commits.

Why Standard RAG Is Insufficient

- Similarity search from a failing test's text mostly retrieves the test and the function under test, not the commit message or changelog entry that states intent.
- The docstrings and old tests it does retrieve describe pre-change behaviour, so the retrieved evidence can point at the wrong hypothesis (stale evidence).
- RAG returns text. It does not decide a direction, run a probe, or check that a repaired test is still strong.

Why a Single-Agent System Is Insufficient

This is what we test, not an assumption. The mechanism we suspect is that the same agent that wants a green build also judges whether editing the test is legitimate (conflicting objectives), and that a "do not weaken tests" instruction gets diluted over a long tool-using trajectory. It also has no independent step that measures test strength after repair. B2 (a single agent with a strong instruction prompt) and A5 (roles merged) are there to challenge this claim.

4. Dataset Availability

Operational / Normal-use Data

- **Real base data: git histories of small, pure-Python, MIT/BSD-style libraries** with fast pytest suites. We choose about 3 for evaluation and 1 for the dev stream. *(Candidates such as toolz, boltons, tabulate or humanize are examples only. We verify licence, test speed, dependency footprint and the number of qualifying commits before committing to any of them, using the GitHub, PyPI and deps.dev APIs.)*
- **RAG corpus per library (real):** README and docs, docstrings, CHANGELOG, and commit messages up to the current commit. It never includes future commits.
- **Synthetic component:** only the injected regressions, generated by mutation testing (for example mutmut) on real code. All code, tests and commit messages are real.

Evaluation Data

Test set of **72 episodes** in 3 held-out libraries (24 per library), run as **chronological streams** so that memory has something to carry over. A dev stream of about 24 episodes in a fourth library is used only for tuning and model selection.

ID	Scenario	Construction	Gold
S1	Outdated test (30 total)	Real commit c that changed source and tests. Apply *only* the source part to its parent and keep the old tests	UPDATE_TEST, with the developer's test diff as reference
S2	Injected regression (12)	Source-only commit where tests still pass, plus one mutant that at least one test kills	FIX_CODE. Tests must stay untouched
S3	Mixed (12)	S1 plus a mutant in a *different* function	Per-test labels: UPDATE_TEST for the outdated ones, FIX_CODE for the new failures

S4	Decoy regression (9)	Later in the stream, a mutant in a function whose change was approved earlier, with a different symptom	FIX_CODE. Memory must not justify an update
S5	Intent-stripped (9)	S1 with an uninformative commit message, docs changes removed, and no memory coverage	ESCALATE
T	Tool faults	About 10% of tool calls fail (patch does not apply, bad test selector, timeout)	Recover without unsafe edits

Difficulty levels: easy (one failing test, one function), medium (several tests, one function), hard (mixed, decoy, or several functions).

Ground Truth

- **Per-failing-test label** (UPDATE_TEST or FIX_CODE), determined by running tests with the source-only change (outdated set) and with the mutant added (regression set).
- **Developer's test diff** as the reference repair for outdated tests.
- **Precomputed mutants per function**, so test strength can be measured deterministically.
- **Gold evidence:** the commit-message sentences and docs/changelog entries that state the intent.
- Stability filter: every test is run 5 times, and unstable tests or episodes are dropped.

Scripted Users / Injected Scenarios

There is no live user. Each episode is a fixed CI event. A **simulated maintainer** reviews the final patch: it approves if the direction matches the gold label and the suite passes, and the approval triggers the memory write. In S5 the maintainer expects an escalation.

Data Construction Difficulty

Moderate to high. The hard part is reproducible environments (pinned Python and dependencies, Docker, no network) and the episode-building scripts (splitting commits into source and test parts, running before and after, mutant selection, stability filtering). Label logic is simple once that is in place. We should expect 3–4 weeks for the environment and construction scripts.

5. Related Work

- **Issue-resolution agents.** SWE-bench (Jimenez et al., 2023), SWE-agent and Agentless evaluate patches by running tests. This assumes the tests are a trustworthy oracle and does not measure whether an agent altered them. *(Analyses of test adequacy in such benchmarks exist. Verify before citing.)*
- **Automated test repair.** TaRGet repairs broken tests with pre-trained code language models and introduces TaRBench, covering 45,373 broken-test repairs across 59 open-source projects. Earlier work such as ReAssert also repaired tests. These works assume the test *should* be repaired. They do not decide between repairing the test and fixing the code.
- **Test/code co-evolution.** Studies such as Zaidman et al. (2011, from memory) examine how tests and production code change together. This motivates our commit-derived episodes.

- **Mutation testing.** Jia & Harman's survey (2011, from memory) and tools such as mutmut give a standard, deterministic way to measure test strength. We use it as an integrity metric.
- **Agent memory with verification.** Voyager stores skills that were verified by execution, and Reflexion and ExpeL store lessons. We store *maintainer-approved behaviour changes with executable probes* and study how broadly they should apply.
- **The PDF.** Direction E (incident response) also has a hypothesis loop with a verifier that rejects unsupported hypotheses. Our project shares that pattern but works on a different task, has a different decision (test vs code) and a different memory design.

Gap and difference. To our knowledge, the test-vs-code decision, with test-integrity measurement and scoped memory across commits, is not covered by the works above. We have limited evidence for novelty beyond that, so we present it as a *design-decision study with a new evaluation construction*, not a new algorithm.

6. Research Question

Primary RQ:

> On chronological streams of CI failures in small Python libraries, does a role-separated pipeline that (i) grounds each repair decision in retrieved change-intent evidence, (ii) remembers maintainer-approved behaviour changes as scoped records with executable probes, and (iii) checks test integrity after repair make fewer wrong-side repairs and weaken tests less than a single tool-using coding agent with the same tools, model and budget?

Secondary RQ:

> Does scoping memory to a function, behaviour signature and executable probe reduce over-generalisation on decoy regressions compared with coarse module-level memory, while still saving tool calls on later covered changes?

7. Hypothesis

Compared with B1 (single tool-using agent, standard RAG, no memory), the full system will have at least **+0.15 higher macro-F1 for the per-test direction decision**, a **masked-regression rate of at most 5%** (against at least 20% for B1), a **mutant-kill retention of at least 0.9** (against at most 0.7), and at least **25% fewer tool calls on episodes covered by an earlier approved change**. Scoped memory will show at most 10% over-generalisation on decoys against at least 30% for coarse memory. The hypothesis is rejected if the paired-bootstrap 95% CI for the direction-F1 difference includes 0, or if B2 closes the gap to under 5 points.

(Thresholds are provisional. We fix them after the dev-stream pilot and before touching the test set. A negative result against B2 is still a valid finding.)

Metrics Supporting the Hypothesis

- Per-test direction accuracy and macro-F1
- Masked-regression rate (test edited when gold is FIX_CODE)
- Reverted-intent rate (source edited when gold is UPDATE_TEST)

- Mutant-kill retention (updated test vs the developer's test)
- Over-generalisation rate on decoys, and tool calls per covered episode

8. Proposed System

Multi-Agent Architecture

Agent	Role	Inputs	Outputs	Tools	Memory
Triage Agent	Runs the suite, groups failures by changed function, reruns to confirm stability	CI event (diff, commit message)	Failure groups with tracebacks and assertion diffs	`run_tests`, `git_show`, `read_file`	Read-only
Intent Analyst	Retrieves change-intent evidence and forms a hypothesis per failure group (outdated test, regression, or unknown) with citations	Failure groups	Hypothesis, cited evidence IDs, predicted new behaviour	`search_repo`, `git_log`, `git_show`, `git_blame`, `memory.read`	Read-only
Repairer	Applies the decision: a test patch for outdated tests, a source patch for regressions	Hypothesis and evidence	Patch in the sandbox worktree, or an escalation request	`apply_patch(scope=src or test)`, `run_tests`, `read_file`	Read-only
Integrity Guard	Independently verifies the outcome and is the only agent that writes memory (after maintainer approval)	Patch, hypothesis, memory record	Approve, reject or escalate, and memory write	`run_tests`, `run_mutants`, `memory.read/write`, `escalate`	Read and write

All four agents run on the **same hosted LLM** (Section 13), so differences between conditions come from the design, not the model.

Why role separation is useful (to be tested, not assumed):

- Evidence gathering is separated from the agent that would benefit from a quick green build.
- Verification is independent, and memory writes are limited to one auditable place.

Overall Workflow

1. A CI event arrives. The Triage Agent runs the suite, reruns failures to confirm they are stable, and groups them by function.
2. For each group, the Intent Analyst retrieves evidence from the commit message, docs, changelog, docstrings, earlier commit messages and memory records. It states a hypothesis and predicts what the new behaviour should be.
3. **Decision policy:** UPDATE_TEST requires a cited piece of positive evidence for an intended change. A memory record counts as evidence only if its probe passes on the current code. REGRESSION needs behaviour that contradicts the evidence or the probe. If neither side has evidence, the outcome is ESCALATE.

4. The Repairer applies a test patch or a source patch within the allowed scope and reruns the tests.
5. The Integrity Guard checks the result. For test patches, it runs the mutants of the function against the updated tests and compares their kill rate with the pre-change tests, and it looks for lost assertions or added skip/xfail markers. For source patches, it checks that no test file was touched and that the suite passes.
6. If the Guard rejects, the Repairer gets one revision. If the Guard still rejects, the outcome is ESCALATE.
7. The simulated maintainer reviews. On approval, the Guard writes a memory record.

Retrieval: a hybrid index (BM25 keyword search plus `voyage-code-3` embeddings, optionally reranked with `rerank-2.5`) over docs, docstrings, changelog and commit history up to the current commit, plus memory records.

Memory (persistent per repository):

- *Stored:* records of the form {function, behaviour signature, direction, executable probe, evidence IDs, commit}. The probe is derived from the assertions in the approved test patch.
- *Updated:* only after maintainer approval. A newer approved change to the same function supersedes the older record.
- *Wrong or stale records:* they are handled by the probe. If the probe fails on the current code, the record is not used as evidence. A record that keeps failing is flagged.

Reasoning: hypothesis testing per failing test (predict the new behaviour from the evidence, then check it), which we score at the step level.

Human escalation: `escalate` with a summary of failing tests, the evidence found and missing, and both hypotheses.

9. Capabilities Used

Capability	Why the problem needs it	How we implement it	How we evaluate it
Agentic abilities (multi-agent, memory, state tracking, hand-off) *— core to the RQ*	The direction decision needs independent verification, and approved changes span many commits	Four roles, scoped memory with probes, write authority only for the Guard, escalation	Per-test direction F1, masked-regression rate, reverted-intent rate, over-generalisation rate, tool calls on covered episodes, escalation precision and recall, memory-record accuracy
RAG *— core to the RQ*	Intent evidence lives in history and docs, and much of the obvious evidence is stale	Hybrid index (BM25 + Voyage code embeddings) over docs, docstrings, changelog and history up to the current commit	Gold-evidence Recall@k, evidence-citation faithfulness (does the cited item exist and support the claim), stale-evidence error rate

Reasoning — *supports the decision*	One CI run can mix outdated tests and real regressions, and each needs its own hypothesis	Per-test hypothesis and predicted behaviour, then a check against the evidence or probe	Per-test hypothesis accuracy, evidence-justified decision rate, distribution of the first wrong step in failed episodes
Tool calling — *supports safe action*	The system must run tests, read history and edit files in a sandbox without collateral damage	The tool set in Section 8, with scope-restricted `apply_patch`, fault injection and a cached mutant set	Tool-call accuracy against reference traces, invalid-argument rate, error-recovery rate, out-of-scope-edit rate, test-weakening rate

Counted as unsafe or unapproved actions:

- Editing files outside `src` and `tests`, such as CI config, `conftest.py` or dependency files.
- Weakening tests: fewer assertions, added skip/xfail, loosened comparisons, or swallowing exceptions.
- Wrong-side edits (masked regression, reverted intent).
- A memory write without maintainer approval.

10. Novelty and Research Contribution

Existing Approach

Test-repair models, issue-resolution agents and mutation testing exist, and agent memory exists in generic forms.

Our Difference

We combine the direction decision, intent-grounded retrieval, verified scoped memory with probes and a mutation-based integrity check in one stream evaluation.

Actual Research Contribution

1. A **benchmark construction** with per-test labels (commit-derived outdated tests, injected regressions, mixed failures, decoys and intent-stripped cases).
2. A **test-integrity metric** (mutant-kill retention plus weakening checks) that goes beyond "tests pass".
3. A controlled study of **memory scope**: probe-backed scoped records against coarse notes, in terms of savings and over-generalisation.

What We Are Experimentally Testing

The design decisions are (a) intent retrieval, (b) memory scope with probes, (c) independent integrity checking and (d) role separation.

11. Evaluation Plan

Evaluation Environment

A Docker sandbox with no network for the repository under test, resource limits, pinned dependencies and a worktree per episode. LLM and embedding calls go out through the orchestrator only, never from inside the sandbox. The agent has no access to future commits or to the developer's reference test diff.

Dataset/Test Set

72 episodes in chronological streams over 3 libraries (S1–S5 plus tool faults), a dev stream of about 24, and difficulty levels as in Section 4.

Baseline

Baseline	Description
B0	Plain LLM given the failing output and the diff, no tools or memory
B1	Single tool-using agent with the same tools, standard top-k RAG over the repo, no memory. This is the main baseline
B2	B1 plus a strong instruction prompt ("edit tests only with evidence of intended change, never weaken assertions"), testing whether prompting alone is enough
Trivial baselines	Always-UPDATE_TEST (maximises green builds) and always-FIX_CODE, to show what the metrics reveal
Oracle	The gold labels

All LLM conditions use the same model, token budget and tool-call budget per episode.

Metrics

- **Task-level:** correct-repair rate (right direction, suite passes, integrity preserved), per-test direction macro-F1, tokens and tool calls per episode.
- **Safety and integrity:** masked-regression rate, reverted-intent rate, test-weakening rate, mutant-kill retention, out-of-scope-edit rate.
- **Escalation:** precision, recall and unnecessary-escalation rate.
- **Statistics:** paired bootstrap and McNemar tests, 2 seeds for core conditions.

Capability-Specific Metrics

As in the Section 9 table. In addition, a **contamination probe**: the plain model is asked to complete each real commit's change without context, to estimate how much it may have memorised.

Ablations

Condition	What it removes	What it tests
Full system	—	Reference

A1: No intent retrieval	Analyst sees only failing output and diff	Value of change-intent evidence
A2: No memory	Memory disabled	Value of cross-commit memory (cost and consistency)
A3: Coarse memory	Module-level notes without probes	Secondary RQ: scope and probes
A4: No Integrity Guard	No mutation check, no independent verification	Value of the integrity step
A5: Single agent, everything else kept	Role separation only	Value of role separation
Model check (subset)	Main model replaced with a second model family	Whether the main finding holds on a different model

A1–A3 and A5 test the primary and secondary RQs directly. A4 and the model check can run on a subset.

Qualitative Analysis

We inspect:

- A masked regression, and how it slipped through.
- A decoy where memory was wrongly reused, and one where the probe correctly blocked it.
- Stale docstrings that led to a wrong hypothesis.
- Cases where the Analyst and the Guard disagreed.
- A repaired test that passes but has a lower kill rate than the developer's.
- Tool failures such as a patch that does not apply.
- Escalations that a human would have skipped.

Failure Analysis

For every failed episode, we log the retrieved evidence, the per-test hypotheses, the patch, the mutation results and the memory reads and writes, and assign one cause: triage error, retrieval or stale evidence, reasoning error, repair error, integrity miss, memory error, tool error or API error (rate limit or provider outage, logged separately and retried, never counted against the agent). We report counts per cause. The primary metrics come from test runs, mutants and labels, not from an LLM judge. Any LLM-judged aspect (for example explanation quality) is secondary and validated against about 100 human labels.

12. Compute Budget

Component	Estimate
Model	One free hosted model for all agents and all conditions: Qwen3 Coder 480B on NVIDIA NIM (final choice made on the dev stream; see Section 13). Mistral Devstral 2 as backup and for the model check.
Local GPU	None. No model runs locally.

API calls	About 15 LLM calls per episode for the full system and about 10 for B1/B2. Roughly 12,000–20,000 calls in total
Token usage	Roughly 50–90M tokens (code contexts, about 4–6k per call)
Embedding tokens	A few million tokens for the RAG corpora of 4 libraries, well within Voyage's 200M free tokens
Approximate API cost	**$0** on free tiers. Runtime about 3–5 days on NIM (~40 requests/min), or about a week on Mistral alone (~2 requests/min, 1B tokens/month)
CPU compute	Test and mutant runs on CPU, about 30–40 CPU-hours in total, parallelisable in Docker
Number of experiments	9 main conditions (B0, B1, B2, Full, A1–A5), 2 seeds for core conditions, A4 and the model check on subsets, 1 dev sweep

13. APIs and Models

All services below are free and need no credit card (checked October 2026; limits change, so confirm on each console after signing up). Every LLM provider exposes an OpenAI-compatible endpoint, so the orchestrator switches providers by changing only `base_url` and the key.

13.1 LLM APIs

Role	Provider	Get key	Base URL	Free limit
Main model, all conditions	NVIDIA NIM	build.nvidia.com → Get API Key	`https://integrate.api.nvidia.com/v1`	~40 requests/min per account
Backup + model check	Mistral (Experiment plan)	console.mistral.ai → API Keys (phone verification)	`https://api.mistral.ai/v1`	~2 requests/min, ~1B tokens/month
Spare (quick tests only)	Groq	console.groq.com → API Keys	`https://api.groq.com/openai/v1`	1,000 requests/day, 200K tokens/day per model
Spare (quick tests only)	Google Gemini	aistudio.google.com → Get API key	`https://generativelanguage.googleapis.com/v1beta/openai/`	~250–1,500 requests/day (Flash)

13.2 Candidate Models

Provider	Model	API ID (verify with `/v1/models`)	Reason
NVIDIA NIM	**Qwen3 Coder 480B** (first choice)	`qwen/qwen3-coder-480b-a35b-instruct`	Built for agentic coding and tool use, 256K context
NVIDIA NIM	DeepSeek V3.2	`deepseek-ai/deepseek-v3.2`	Strong at code and reasoning

NVIDIA NIM	GPT-OSS 120B	`openai/gpt-oss-120b`	Reliable tool calling, fast
Mistral	**Devstral 2** (backup)	`devstral-2512`	Agentic coding model
Mistral	Codestral	`codestral-2508`	Code correction and test generation
Mistral	Mistral Large 3	`mistral-large-2512`	Second-model check on a subset
Groq	GPT-OSS 120B	`openai/gpt-oss-120b`	Spare

Selection: run the three NIM candidates on the dev stream and keep the one with the most reliable tool calling. Freeze it, by dated model ID, before touching the test set. Use dated Mistral IDs rather than `-latest` aliases so the model cannot change mid-experiment.

13.3 Embedding and Reranking API

Provider	Get key	Model	Free	Use
Voyage AI	dash.voyageai.com → API Keys	`voyage-code-3`	200M tokens	Embeddings for docs, docstrings, changelog, commit messages, memory records
Voyage AI	same key	`rerank-2.5`	200M tokens	Optional reranking of hybrid retrieval results

The keyword half of the hybrid index uses BM25 (the `rank_bm25` Python library), which needs no API.

13.4 Data APIs for Episode Construction

API	Key	Use
GitHub REST / GraphQL	GitHub → Settings → Developer settings → Personal access token (read-only, public repos)	Commit histories, commit messages, licences, candidate library search (5,000 requests/hour)
PyPI JSON API	None (`https://pypi.org/pypi/<pkg>/json`)	Check that a library is pure Python and inspect its dependencies
deps.dev API	None	Dependency footprint and licence
Libraries.io (optional)	libraries.io/account	Finding candidate small libraries

13.5 Usage Rules

- **One model for all conditions.** Calls are never spread across providers to gain capacity, because that would make baselines and the full system incomparable.
- **Retry with exponential backoff and a response cache** keyed on (model, prompt, seed). Rate-limit errors are logged separately from injected tool faults (scenario T).
- **Log the exact model ID and date of every call.** Run the core experiments early in the semester, because free tiers can change without notice (GitHub Models was retired and Cerebras ended its free tier in 2026).

- **Keys live in a `.env` file listed in `.gitignore`** and are never committed.
- Free-tier prompts may be used for provider training (Mistral, Gemini). This is acceptable because all code and data are from open-source repositories.

```
NVIDIA_API_KEY=
MISTRAL_API_KEY=
VOYAGE_API_KEY=
GITHUB_TOKEN=
# optional spares
GROQ_API_KEY=
GEMINI_API_KEY=
```

14. Risks and Fallback Plan

Main Risks

- **Environment reproducibility:** dependencies, Python versions and unstable tests can consume weeks.
- **Too few qualifying commits** in small libraries.
- **Contamination:** popular open-source libraries may be in pretraining data.
- **Artificial faults:** mutants are not real regressions, and S5 assumes that "no evidence means escalate".
- **Weak or strong baseline:** B2 might match the full system.
- **Mutation score as a proxy** for test strength.
- **Free API availability:** rate limits may tighten, or a model or free tier may be withdrawn mid-semester.
- **Tool-calling reliability** of free open-weight models.
- **Scope creep:** four agents and memory with probes.

Fallback

- Pick libraries with no compiled dependencies and fast tests, and drop episodes that are unstable.
- Use 4 libraries with shorter streams, or relax to commits that changed source and tests even by small diffs.
- Prefer lesser-known libraries, report the contamination probe, and rely on the injected mutants, which are novel by construction.
- *(Rec.)* Add a small best-effort real-regression sanity set from history (for example "fix regression" commits, about 10–15 episodes), and report S5 separately as a secondary result.
- Report negative results against B2 openly, and add a second integrity signal (assertion-count and skip/xfail checks) next to the mutation score.
- If NIM becomes unavailable, switch every condition to Mistral Devstral 2 and rerun the affected conditions. Never mix models within one comparison.
- Choose the main model on the dev stream partly by tool-calling reliability, and keep tool schemas simple.
- If time is short, merge Triage into the Analyst (three agents), and drop A4.

15. Feasibility

Realistic for a semester, with the environment and dataset construction as the main engineering effort. The domain is familiar, tools run in a sandbox, labels come from executed tests and mutants, and the LLM and embedding costs are zero on free tiers. The riskiest parts are reproducible environments, contamination and free-tier availability, addressed by library selection, the stability filter, the contamination probe, the injected mutants and a configured backup provider. The orchestration can be a Python state machine with Docker.

Verification Checks

RQ Check

- Experimentally testable? **YES**
- Compares a specific method against a baseline? **YES**
- Measurable outcomes? **YES**
- Small controlled dataset sufficient? **YES**
- Directly motivates the architecture? **YES**
- Answerable within compute budget? **YES** (free APIs, Section 13)

Capability Check

- Multiple cooperating agents: **YES**
- Persistent memory: **YES**
- Retrieval/RAG: **YES**
- Tool usage: **YES**
- Reasoning: **YES**

Each has a problem-specific reason, an implementation and a metric in Section 9.

Evaluation Strength Check

- Controlled evaluation set: **YES**
- Known ground truth: **YES**
- At least one baseline: **YES**
- Quantitative metrics: **YES**
- Capability-specific metrics: **YES**
- Capability ablations: **YES**
- Tool safety evaluation: **YES**
- Qualitative difficult cases: **YES**
- Failure analysis: **YES**
- LLM-as-judge avoided or human-validated: **YES**

Appendix A. Fine-Tuning Plan (Optional Extension)

Fine-tuning is kept outside the core study so that the main results do not depend on it. If time allows, we add it as one extra capability with its own ablation. All training and serving happens on free cloud GPUs; nothing runs on a local machine.

A.1 What Is Fine-Tuned and Why

- **Task:** only the Repairer's **UPDATE_TEST patch generation**. Given the failing test, the source diff, the traceback and the cited intent evidence, the model writes the updated test.
- **Why this subtask:** it is narrow and well defined, and real training data is plentiful (developers' own test diffs). The direction decision is *not* fine-tuned; it stays with the Intent Analyst and the Integrity Guard on the main hosted model, so the core RQ is unaffected.
- **Capability claim:** a small tuned model can write test patches that pass and keep mutant-kill strength as well as a much larger prompted model.

A.2 Training Data

Item	Plan
Source	About 8 small pure-Python libraries that are **disjoint from the 3 test libraries and the dev library**, mined with the GitHub API
Example	A triple (source diff, failing test + traceback, developer's test diff) from commits that changed source and tests together and where the old test fails on the new source
Input format	Prompt = failing test, source diff, traceback, commit message. Target = the developer's updated test (unified diff)
Size	About 1,500–3,000 triples if enough qualifying commits exist; 90/10 train/validation split by repository, not by example
Filtering	Keep only triples where the developer's test passes on the new source; drop tests that are unstable over 5 runs; drop diffs that only delete or skip tests (so the model never learns to weaken tests)
Decontamination	Exact and near-duplicate check (function names and normalised test bodies) against all evaluation libraries
Storage	Private dataset on the Hugging Face Hub (or a Kaggle Dataset)

A.3 Base Model and Method

Item	Choice
Base model	**Qwen2.5-Coder-7B-Instruct**, using the Unsloth 4-bit build `unsloth/qwen2.5-coder-7b-instruct-bnb-4bit`. Fallback if memory is tight: Qwen2.5-Coder-3B-Instruct
Method	QLoRA (4-bit base, LoRA adapters) with Unsloth and TRL's `SFTTrainer`, loss on the target tokens only
Starting hyperparameters	LoRA rank 16, alpha 16–32, dropout 0, learning rate 2e-4 with cosine decay, 2–3 epochs, max sequence length 4,096, effective batch size 16 (gradient accumulation), seed fixed

Run time	About 1–3 hours per run on a free T4/P100; at most about 8 GPU-hours including a small sweep
Model selection	Validation loss plus test-patch pass rate on the validation repositories; never on the test libraries
Output	LoRA adapter (about 100–200 MB) pushed to a private Hugging Face repository, with the training config and data hash

A.4 Serving the Tuned Model Without a Local Machine

No free hosted API serves a custom LoRA on Qwen2.5-Coder (Cloudflare Workers AI's free LoRA beta supports only a few older base models, and Together AI and similar services are paid). We therefore serve it ourselves on a free cloud GPU:

1. Start a Kaggle notebook with a GPU, install **vLLM**, and load the base model with the LoRA adapter from the Hugging Face Hub.
2. vLLM exposes an **OpenAI-compatible endpoint**, so the orchestrator's Repairer calls it exactly like NIM, by changing only `base_url`.
3. Reach the endpoint from the orchestrator through a free tunnel (for example a Cloudflare quick tunnel). Check Kaggle's current terms of use before relying on this; if it is not allowed, run the ablation episodes inside the notebook instead, with the pinned library environment installed in a virtual environment.
4. Sessions last up to about 9–12 hours, so the swap ablation runs in batches with the response cache and per-episode checkpoints.

A.5 Evaluation of Fine-Tuning

Condition	Repairer for UPDATE_TEST patches	What it tests
Full system (reference)	Main hosted model (e.g. Qwen3 Coder 480B on NIM)	Reference
FT: Tuned 7B	Qwen2.5-Coder-7B + LoRA	Does a small tuned model match the large prompted model?
FT-control: Base 7B	Qwen2.5-Coder-7B without the adapter, same prompt	How much the gain comes from tuning rather than the model itself

- Only the Repairer's test patches change. Triage, the Intent Analyst and the Integrity Guard stay on the main model, so the decision metrics are unchanged by design.
- Run on the UPDATE_TEST episodes (S1 and the outdated tests in S3) of the held-out libraries.
- **Metrics:** test-patch pass rate, mutant-kill retention against the developer's test, test-weakening rate (lost assertions, added skip/xfail, loosened comparisons), out-of-scope-edit rate, patch-apply failure rate, and tokens and latency per patch.
- **Statistics:** paired bootstrap on the same episodes, 2 seeds.
- If fine-tuning does not pay off, we report the negative result.

A.6 Things Required for Fine-Tuning

Accounts and keys (all free):

Service	What to do	Used for
Kaggle	Sign up at kaggle.com, verify a phone number (required to enable GPUs), set the notebook accelerator to GPU T4 x2 or P100	Training and serving. About 30 GPU-hours/week free, sessions up to about 9–12 hours, 20 GB persistent storage
Hugging Face	Sign up at huggingface.co, create an access token with write permission, accept the model licence if asked	Downloading the base model, storing the private dataset and the LoRA adapter
GitHub token	The same read-only token as in Section 13	Mining the training libraries
Google Colab (backup)	Sign in with a Google account, select a T4 runtime	Backup GPU if the Kaggle quota runs out (sessions can be cut off)
Weights & Biases (optional)	Free personal or academic account, API key	Logging training curves and runs

Python libraries (installed in the Kaggle notebook):

Library	Purpose
`unsloth`	Fast, memory-efficient QLoRA training on a 16 GB GPU
`transformers`, `trl`, `peft`	Model loading, `SFTTrainer`, LoRA adapters
`bitsandbytes`	4-bit quantisation
`datasets`, `huggingface_hub`	Loading the triples and pushing the adapter
`vllm`	Serving the base model plus LoRA through an OpenAI-compatible endpoint
`PyDriller` or `GitPython`	Splitting commits into source and test parts when building the triples
`pytest`	Checking that each developer test passes on the new source (data filtering)

Other things to prepare:

- A list of about 8 candidate training libraries, checked with the PyPI and deps.dev APIs to be pure Python, MIT/BSD-style and disjoint from the evaluation libraries.
- A triple-mining script and a fixed prompt template shared by training and inference.
- A training notebook (Unsloth QLoRA) and a serving notebook (vLLM with the adapter), both saved with fixed seeds and pinned library versions.
- A record of the base model revision, the adapter commit on Hugging Face, the data hash and the hyperparameters, so the results can be reproduced.

```
HF_TOKEN=          # Hugging Face, write access
KAGGLE_USERNAME=   # only if using the Kaggle API from scripts
KAGGLE_KEY=
WANDB_API_KEY=     # optional
FT_BASE_URL=       # vLLM endpoint for the tuned Repairer
```

A.7 Time and Risks

- **Time:** about 1 week to mine and filter the triples (mostly reusing the episode-construction scripts), and about 1 week to train, serve and evaluate.
- **Too few triples:** relax to commits with small test diffs, or add more training libraries.
- **Kaggle quota or session limits:** train in one session per run, checkpoint often, use Colab as backup.
- **Serving is awkward on a notebook:** run the swap ablation on a subset and in batches.
- **Learning to weaken tests:** prevented by filtering the data and measured by the test-weakening rate.
- If time is short, fine-tuning is the first thing to drop, before any of the cuts in Section 14.